

Algoritmi di ordinamento

Saturday, April 25, 2026 11:20 AM

Esistono degli algoritmi di ordinamento che si possono classificare in base alla complessità:

- $O(n^2)$: semplici, iterativi, basati sul confronto
- $O(n^x)$ con $x < 2$: evoluzione di quelli quadratici classici ma sempre basati sul confronto
- $O(n \log n)$: più complessi, ricorsivi, basati sul confronto.
- $O(n)$: applicabili solo con ipotesi restrittive sui dati, basati sul calcolo.

Si può effettuare un'analisi più raffinata, basata su:

- Scambi
- Confronto

Non esistono algoritmi di ordinamento basati sul confronto la cui complessità asintotica di caso peggiore sia migliore di quella lineare. Infatti questi sono considerati ottimi.

ALGORITMI DI ORDINAMENTO ITERATIVI QUADRATICI

1) Insertion sort

Siano dati N interi memorizzati in vettore A con indici compresi tra $l=0$ e $r=N-1$.

Concettualmente A è diviso in due sottovettori:

- Di sinistra già ordinato che contiene un solo elemento.
- Di destra da ordinare che contiene $N-1$ elementi.

Al passo i -esimo si colloca nel sottovettore di sinistra, nella posizione corretta l'elemento $x=A_i$:

- Si scandisce il sottovettore di sinistra fino a trovare un elemento per cui $A_j > A_i$.
- Durante la scansione si opera uno scorrimento a destra di una posizione degli elementi da A_j a A_{i-1} .

```
void InsertionSort(int A[], int N){
    int i, l=0, r=N-1, x;
    for(i=l+1; i<=r; i++){
        x = A[i];
        j = i-1;
        while(j >= 0 && x < A[j]){
            A[j+1] = A[j];
            j--;
        }
        A[j+1] = x;
    }
}
```

2) Bubble/Exchange Sort

Siano dati N interi in un vettore A con indici compresi tra $l=0$ e $r=N-1$.

Il vettore A è diviso in due sottovettori:

- Quello di destra già ordinato poiché vuoto.
- Quello di sinistra da ordinare e contiene N elementi.

L'operazione elementare è un confronto tra l'elemento corrente e il suo successivo.

Al passo i -esimo si identifica il massimo del sottovettore sinistro:

- Si scandisce il sottovettore di sinistra confrontando le coppie $A[j]$ e $A[j+1]$ e scambiandole se il primo è maggiore del secondo.

```
void BubbleSort(int A[], int N){
    int i, l=0, r=N-1, x;
    int temp;
    for(i=l; i<r; i++){
        for(j=l; j<r-i+1; j++){
            if(A[j] > A[j+1]){
                temp = A[j];
                A[j] = A[j+1];
                A[j+1] = temp;
            }
        }
    }
}
```

```

        A[j+1] = temp;
    }
}
return;
}

```

E' possibile un'ottimizzazione del seguente algoritmo con un flag che indica se ci sono stati scambi, in modo che l'esecuzione prosegua se ci sono stati:

- Il ciclo esterno viene eseguito al massimo N-1 volte
- Il ciclo interno all'i-esima iterazione viene eseguito sempre N-1-I volte.

Si migliora il caso medio ma non cambia la complessità peggiore.

```

void BubbleSort(int A[], int N){
    int i, l=0, r=N-1, x;
    int temp;
    int flag = 1;
    for(i=l; i<r && flag == 1; i++){
        for(j=l; j<r-i+1; j++){
            if(A[j] > A[j+1]){
                flag = 1;
                temp = A[j];
                A[j] = A[j+1];
                A[j+1] = temp;
            }
        }
    }
    return;
}

```

3) Selection sort

Siano dati N interi in A con indici compresi tra l e r. il vettore A e' diviso in due sottovettori:

- Quello di sinistra gia' ordinato vuoto
- Quello di destra disordinato.

Al passo i-esimo si identifica il minimo del sottovettore destro, si scandisce il sottovettore di destra, ipotizzando il minimo in A[i] e aggiornando il minimo ad ogni confronto con gli elementi successivi. Quando il ciclo termina e' stato identificato il minimo tramite la sua posizione e lo si scambia con A[i].

```

void SelectionSort(int A[], int N){
    int i, j, l=0, r=N-1, min;
    int temp;
    for(i=l; i<r; i++){
        min = i;
        for(j = i+1; j <= r; j++){
            if(A[j] < A[min]){
                min = j;
            }
        }
        if (min != i){
            temp = A[i];
            A[i] = A[min];
            A[min] = temp;
        }
    }
}

```

4) Shell sort

Si impegna a superare uno dei limiti dell'Insertion Sort: il confronto avviene solo tra elementi adiacenti.

Lo shell sort confronta ed eventualmente scambia elementi a distanza h tra di loro, definendo una sequenza decrescente di interi h che termina con 1, riconducendosi all'ultimo passo all'insertion sort.

Si considerano le sottosequenze formate dagli elementi del vettore I cui indici distano h . Se le sottosequenze sono ordinate, il vettore si dice h -ordinato. Una volta identificate le sottosequenze, per ognuna di esse si applica l'insertion sort.

```
for(i = l+h; i <= r; i++){
    j = i;
    x = A[i];
    while(j >= l+h && x < A[j-h]){
        j -= h;
    }
    A[j] = x;
}
```

Le possibili sottosequenze sono:

- Shell: $h_i = 2^{i-1}$
- Knuth: con $h_0=0$ inizialmente e $h_i = 3h_{i-1} + 1$

ALGORITMI DI ORDINAMENTO ITERATIVI LINEARI

1) Counting sort

Si procede per calcolo e non per confronto. Per ciascun elemento da ordinare x si calcolano quanti elementi lo precedono nell'ordinamento. Prima si calcolano le occorrenze semplici di x , cioè quante istanze di x compaiono nell'input. A partire dalle occorrenze semplici si calcolano le occorrenze multiple, cioè quanti elementi sono $\leq x$.

Si usano 3 vettori:

- A di input
- B , vettore risultato
- C , vettore delle occorrenze semplici/multiple di k interi se I dati sono nell'intervallo $[0 \dots k-1]$.

Per il calcolo delle occorrenze multiple si procede nel seguente modo: scansione del vettore C delle occorrenze semplici

- In $C[0]$ sono memorizzate le occorrenze di 0 e di tutti I dati che lo precedono
- Le occorrenze di dati che precedono I sono memorizzate in $C[i-1]$
- Le occorrenze di dati che precedono o sono uguali a I si calcolano come $C[i] = C[i-1] + C[i]$

```
for(i=1; i<k;i++){
    C[i] += C[i-1]
}
```

Per calcolare le posizioni finali:

Scansione del vettore A da destra a sinistra

- In $C[A[i]]$ sono memorizzate le occorrenze multiple di $A[i]$ e di tutti I dati che lo precedono
- La posizione finale in B di $A[i]$ è l'indice $C[A[i]]-1$.
- Una volta memorizzato $A[i]$ nella posizione finale si deve aggiornare il vettore delle occorrenze multiple C all'indice $A[i]$ decrementandolo di 1.

2) Radix sort

I dati da ordinare sono composti da d campi, I cui valori appartengono a un insieme di cardinalità n . esistono due tipi di ordinamento:

- Intuitivo: si ordina secondo la colonna più a sinistra, poi secondo la colonna immediatamente a destra fino ad ordinare secondo la colonna più a destra. Il risultato non è comunque ordinato se non si usa la ricorsione.
- Controintuitivo: si ordina secondo la colonna più a destra, poi secondo la colonna immediatamente a sinistra, fino ad ordinare secondo la colonna più a sinistra. Il risultato risulta essere ordinato.

l'unico vincolo è che l'ordinamento usato su ogni colonna deve essere stabile, per cui si può considerare il Counting Sort.

ALGORITMI DI ORDINAMENTO ITERATIVI LINEARITMICI

L'unica versione iterativa è il Bottom-Up Merge Sort.

1) Bottom Up Merge Sort

Consiste in fondere due sottovettori ordinati in un vettore ordinato la cui dimensione è la somma delle dimensioni dei 2 sottovettori di partenza. La dimensione del vettore di partenza deve essere un multiplo di 2.

Esempio:

Partendo da sottovettori di lunghezza 1 si applica Merge per ottenere ad ogni passo vettori ordinati di lunghezza m doppia. Serve un vettore di appoggio di dimensione N per contenere il risultato del merge. La terminazione consiste nell'avere il vettore di dimensione pari a quello di partenza.

```
Void BottomUpMergeSort(int A[], int B[], int N){
    Int I, q, m, l=0, r=N-1;
    For(m=1; m<=r-l; m= m+m){
        For(i=l; I <=r-m; i+=m+m){
            Q = i+m-1;
            Merge(A, B, I, q, r);
        }
    }
}
```

2) 2-Way Merge

La dimensione del vettore di partenza è un multiplo di 2. si chiama 2-way perché consiste nella fusione di due sottovettori di A ordinati di dimensione m per ottenere un vettore finale di dimensione $2m$. Si può generalizzare a k vettori. l'indice q serve per dividere i due sottovettori a metà.

Approccio:

- Scorrere i sottovettori sinistro e destro mediante gli indici I e j e scorrere il vettore B mediante l'indice k .
- Se è esaurito il sottovettore sinistro, ricopiare gli elementi rimanenti nel sottovettore destro di B .
- Se è finito il sottovettore destro, ricopiare gli elementi nel sottovettore sinistro di B .
- Altrimenti confrontare l'elemento corrente $A[i]$ del sottovettore sinistro con quello $A[j]$ del destro.
- Se $A[i] \leq A[j]$, ricopiare $A[i]$ in B e avanzare I , j resta così.
- Altrimenti ricopiare $A[j]$ in B e avanzare j , I resta invariato.

```
Void Merge(int A[], int B[], int l, int r, int q){
    Int I, j, k;
    i=l;
    j=q+1;
    For(k=l; k<=r; k++){
        If(i>q)
            B[k] = A[j++];
        Else if(j>r)
            B[k] = A[i++];
        Else if((A[i] < A[j]) || (A[i] == A[j]))
            B[k] = A[j++];
        Else
            B[k] = A[i++];
    }
    For(k=l; k<=r; k++){
        A[k] = B[k];
    }
}
```

Per poter generalizzare a N qualsiasi, non necessariamente multiplo di 2, l'estremo destro del

vettore nel Merge e il più piccolo valore tra r e il valore che si avrebbe se la lunghezza fosse una potenza di 2.

```
Void BottomUpMergeSort(int A[], int B[], int N){
    Int I, q, m, l=0, r=N-1;
    For(m=1; m<=r-l; m= m+m){
        For(i=l; I <=r-m; i+=m+m){
            Q = i+m-1;
            Merge(A, B, I, q, min(i+m+m-1, r));
        }
    }
}
```